

# What can you do using a coding agent?

08 May 2026

PML Version:

[https://docs.google.com/document/d/1VurggAe\\_qZlMieK8pA4S2yJjWBd7cnoO8cuvh4zmNZs/edit?tab=t.0](https://docs.google.com/document/d/1VurggAe_qZlMieK8pA4S2yJjWBd7cnoO8cuvh4zmNZs/edit?tab=t.0)

OCL Version:

[https://docs.google.com/document/d/1Y23tu2ePPexkBhh\\_G0RCK1fNio\\_NQ3EZuTbgWa\\_UyPA/edit?tab=t.0](https://docs.google.com/document/d/1Y23tu2ePPexkBhh_G0RCK1fNio_NQ3EZuTbgWa_UyPA/edit?tab=t.0)

This document is not a test. It is a list of things that are reasonably possible to be done today using the best coding agents.

Use this document to self-determine whether you can independently use coding agents to understand unfamiliar repos, produce concrete artifacts, build small systems across languages, parallelize work safely, and verify their output.

I think that even today, the best users of these coding agents are not using them to the best possibilities. I know some companies have individuals consuming \$5000 per day in tokens (knowledge workers). In our company itself, I know some engineering team members have had a consistent 50 commits a day to their repositories in stretches while others have added more than 20k lines of code daily to working, production grade code.

While these extremes are great bragging grounds and i am sure we will all have ability to get to very productive uses of this tech, in this document, i am listing some things that we can say, we all ought to be able to do.

So read this as a motivation, a nudge and not a test. I would love to help every one in this channel to cross this skill journey (and so will many others). So why not start somewhere – ask questions - to each other - or to your favourite coding agent and get going :)

## Eval: Basics

| Category         | Question                                                                                                     | Self-Eval (yes / no) |
|------------------|--------------------------------------------------------------------------------------------------------------|----------------------|
| Repo discovery   | Can you inspect an unfamiliar repo and produce a class/service/module inventory in 30 minutes?               |                      |
| Data model       | Can you produce an ER diagram from code, migrations, ORM models, or schema files in 45 minutes?              |                      |
| API mapping      | Can you map all API endpoints to handlers/controllers in 30 minutes?                                         |                      |
| Flow tracing     | Can you trace one endpoint/event/cron flow end-to-end across files in 45 minutes?                            |                      |
| Testing          | Can you identify the test framework, relevant test files, and exact test command for a module in 30 minutes? |                      |
| Greenfield build | Can you build a small FastAPI service from scratch with tests in 60 minutes?                                 |                      |
| Node.js build    | Can you build a small Node.js service/CLI with tests in 60 minutes?                                          |                      |
| Rust build       | Can you build a small Rust CLI/library with tests in 60 minutes?                                             |                      |
| Parallel work    | Can you split a task across multiple worktrees or agent sessions safely?                                     |                      |
| Verification     | Can you separate what the agent suggested from what you manually verified?                                   |                      |

## Eval: repo reader and simple builder

### B1. Repo artifact inventory

In 30 minutes, can you find major classes, interfaces, services, controllers, models, repositories, jobs, consumers, configs, and utilities.

### B2. API endpoint map

In 30 minutes, can you identify every externally exposed API route or frontend route.

### B3. Test discovery and execution

In 15 minutes, can you find the test framework, relevant test files, and exact commands to run tests. Show the following:

- Test framework and config file.
- Relevant test files.
- Exact commands.
- Actual command result.
- Any failure and interpretation.

### B4. FastAPI greenfield service

In an hour, can you build a small Python FastAPI service from scratch and prove it runs. Show:

- FastAPI app.
- POST /transactions.
- GET /transactions.
- GET /balance.
- Input validation.
- At least 3 tests.
- README with install/run/test commands.

### B5. Node.js greenfield API or CLI

Can you now do the same project in nodejs api or cli ?

### B6. Rust greenfield

Can you do this in rust and show:

- Cargo project.
- CLI accepts file path.
- Counts INFO/WARN/ERROR.
- Handles missing file gracefully.
- At least 3 tests.
- README with cargo commands.



## Eval: Intermediate - repo operator and polyglot builder

Intermediate tasks test whether the engineer can operate safely in unfamiliar repos, produce data/flow diagrams, and connect multiple components.

### I1. ER diagram from repo

In 45 minutes, can you build an ER diagram for all tables and entities in this repo? Use only the repo as your source and cite the source file for every table, entity, and relationship. Show:

- List of tables and entities.
- Primary keys.
- Foreign keys or inferred relationships.
- Source file path for each claim.
- Valid Mermaid ER diagram.

### I2. End-to-end flow trace

In 45 minutes, can you trace one endpoint, event, or cron job end-to-end? Show every major file and function from entry point to the final DB, API, or queue side effect. Show:

- Entry point.
- Step-by-step file and function path.
- External dependencies.
- DB, API, or queue side effects.
- Sequence diagram.
- Known uncertainty.

### I3. Small safe change in unfamiliar repo

In 60 minutes, can you make a small, focused change in an unfamiliar module, keep the diff minimal, and add or update a relevant test? Show:

- Diff or branch.
- Files changed.
- Why these files.
- Test command and result.
- Risk assessment.
- What the agent suggested vs what was manually verified.

### I4. Polyglot service pair: FastAPI plus Node client

In 90 minutes, can you build a simple two-component system — a FastAPI service with a `/convert` endpoint and a Node.js CLI client that calls it? Use hardcoded rates, include tests, and a README with run instructions. Show:

- FastAPI service with `/convert` endpoint.
- Node.js CLI client that calls the service.
- Input validation.
- Tests for the service.
- Tests or scripted verification for the client.
- README with two-terminal run instructions.

## **15. Dockerize and run**

In 60 minutes, can you containerise an existing or new service so it builds and runs cleanly in Docker?

Show:

- Dockerfile.
- Proof it builds.
- Container runs and the service responds.
- Health check or curl proof.
- README with docker build and run commands.

## **16. Bug diagnosis with agent**

In 60 minutes, can you diagnose a seeded bug in an unfamiliar repo — reproduce it, identify the root cause with file paths, fix it, and show the verification command? Show:

- Reproduction steps.
- Root cause with file paths.
- Minimal fix.
- Verification command and result.
- What the agent suggested vs what was manually verified.

## Eval: Advanced - parallel agent operator and system builder

Advanced tasks test whether the engineer can supervise agents, split work safely, reconcile parallel branches, and build larger multi-language systems.

### A1. Multi-worktree parallel plan

In 45 minutes, can you take one feature or analysis task and split it safely into parallel worktrees or agent sessions, without creating merge chaos? Show:

- Task decomposition.
- Worktree or branch names.
- Agent prompt for each lane.
- Shared constraints.
- Merge order.
- Conflict or risk plan.
- Verification plan.

### A2. Execute two parallel worktrees

In 90 minutes, can you actually create two parallel worktrees or branches, make independent changes in each, then reconcile them cleanly? Show:

- Commands used to create the worktrees.
- Branch or worktree names.
- Separate outputs from each lane.
- Final merge or reconcile steps.
- Test result.
- Conflict notes.

### A3. Polyglot mini-system: FastAPI, Node worker, Rust engine

In 150 minutes, can you build a mini fraud-score system across Python, Node.js, and Rust? FastAPI accepts transactions, a Node.js worker processes them, and a Rust CLI or library computes a risk score. Include tests and a README. Show:

- FastAPI ingestion endpoint.
- Node.js worker process.
- Rust scoring CLI or library.
- Clear data contract between components.
- Tests for core scoring and at least one integration path.
- README with run order.

### A4. Repository modernization plan with executable first step

In 90 minutes, can you analyse a repo for modernization opportunities, prioritise them, and then implement the single highest-value, lowest-risk first step? Show:

- Findings with evidence from files or configs.
- Prioritised plan.
- First step implemented.
- Verification (build, tests, or lint).

- Rollback notes.

### **A5. Agent code review and adversarial verification**

In 60 minutes, can you review an agent-generated PR, find correctness, security, test, performance, and maintainability issues, and propose fixes? Show:

- Issue list.
- Severity and blocking or non-blocking classification.
- Suggested fix for each issue.
- Test or verification steps.

### **A6. Performance profiling and targeted improvement**

In 90 minutes, can you find a real performance bottleneck in a small service or script and make a measurable, minimal improvement without doing a broad rewrite? Show:

- Baseline measurement with method and numbers.
- Profiling approach and what the profile showed.
- Short explanation of the bottleneck.
- Targeted code change kept small and focused.
- After measurement showing the improvement.
- Tests or checks proving behavior is unchanged.

## **Eval: Infra and DevOps**

Infra and DevOps tasks test whether the agent can produce verifiable platform artifacts — config files, manifests, pipelines — that not only parse, but actually run end-to-end with proof. Prefer local or dry-run verification over real cloud spend, and lean on commands and outputs rather than prose claims.

### **D1. Terraform plan for a small service**

In 60 minutes, can you write Terraform for a small service (for example an S3 bucket plus a Lambda plus an API Gateway, or a GCS bucket plus Cloud Run) that passes terraform validate and produces a clean terraform plan against a local or test backend? Show:

- .tf files and variables.
- Provider and backend configuration.
- Terraform validate output.
- Terraform plan output.
- README with apply and destroy commands.

### **D2. docker-compose stack from scratch with end-to-end tests**

In 90 minutes, can you stand up a multi-service stack (API plus database plus a worker) with docker-compose, seed data, and a script that runs the full test suite end-to-end against the running stack? Show:

- Docker-compose.yml and per-service Dockerfiles.
- Seed or fixture script.
- One-command test run output (all green).



- Logs proving the services actually talked to each other.
- Teardown command and a clean re-up from zero.

### **D3. CI pipeline that lints, tests, and builds an image**

In 45 minutes, can you write a CI workflow (GitHub Actions or GitLab CI) that on every push lints, runs tests, and builds and tags a container image, and produces a green run? Show:

- Workflow YAML.
- Cache and matrix configuration if relevant.
- Link to or local act run of a passing pipeline.
- Failure mode demo: a deliberately broken commit failing the pipeline.

### **D4. Kubernetes manifests verified on a local cluster**

In 60 minutes, can you write Kubernetes manifests (Deployment, Service, ConfigMap, optional Ingress) for an existing service, validate them with a dry-run or kubeval, and bring the service up on kind or minikube with a curl proof? Show:

- Manifest YAML files.
- Dry-run or kubeval output.
- kubectl apply output on a local cluster.
- curl or port-forward proof the service responds.
- README with up and down commands.

### **D5. Reproducible dev environment from a fresh clone**

In 45 minutes, can you make an existing repo bootstrap from a fresh clone with a single command (devcontainer.json, Nix flake, or Makefile plus asdf or mise) such that tests pass on a clean machine? Show:

- Bootstrap config files.
- The single command and its full output.
- A passing test run.
- Notes on what was previously implicit (system packages, env vars, versions).

### **D6. Observability bolt-on with metrics and a dashboard**

In 60 minutes, can you add structured logging and a /metrics endpoint to a small service, then stand up a Prometheus plus Grafana compose stack and wire one working dashboard panel against real traffic? Show:

- Code diff adding logs and metrics.
- docker-compose for Prometheus and Grafana with provisioned datasource and dashboard.
- A load script that generates traffic.
- Screenshot or JSON of the dashboard panel showing live data.
- README with run order.